

Diff-GNN: Differentiable Graph Neural Networks for Joint Hardware-Software Partitioning and Scheduling

Anonymous Author(s)

Abstract

Sid: Each paper must be no more than 8 pages (including the abstract, figures, and tables), double-columned, 9pt or 10pt font. One page of references is allowed, which does not count towards this 8-page limitation. Sid: (placeholder)

1 Introduction

Our contributions are as follows:

- We present Diff-GNN, a differentiable framework for jointly optimizing HW/SW partitioning and software-task ordering under hardware area constraints.
- We introduce a differentiable ordering head based on Sinkhorn relaxation, which enables software execution priorities to be learned jointly with placement decisions.
- We develop an order-aware soft scheduling surrogate that captures precedence, communication, and software serialization, enabling end-to-end optimization toward makespan minimization.
- We design a discrete inference pipeline with decoding, feasibility repair, and refinement, which produces valid solutions under area constraints and improves robustness to relaxation error.
- We evaluate Diff-GNN against exact optimization, metaheuristic, and prior learning-based baselines on task graphs derived from neural network workloads, demonstrating competitive solution quality and improved scalability.

2 Related Works

- GCN-based HW/SW partitioning 2021 [6]
- HW/SW partitioning survey 2019 [3], HW/SW partitioning survey 2012 [4]
- Arato [1]
- Family of exact solutions includes Dynamic Programming (DP) [], Integer Linear Programming (ILP) [], Branch and Bound (BB) [].
- The common metaheuristics discussed in the literature for HW/SW partitioning are Genetic Algorithms [], Simulated Annealing [], and Tabu Search (TS) []. These works for graph with small number of nodes.
- ILP limitation is ≤ 400 nodes task graph [5]. Ant colony optimization (ACO) [], artificial bees [], particle swarm optimization (PSO) [], artificial immune algorithm []. Due to the complexity of the problem and the limitations of runtime, it is almost impossible for these metaheuristics to achieve global optimization [3].
- Another category of general heuristics is to reduce this HW/SW partitioning problem into another classical NP hard problems, such as graph partitioning [1], or Knapsack problem []

The HW/SW partitioning problem can be formalized as a bi-partitioning problem to find the global optimal solution [2].

3 Problem Setup

We consider a target architecture consisting of a single software processor and multiple hardware (ASIC/FPGA) components. Software tasks execute sequentially on the CPU, while hardware tasks without dependencies can execute in parallel. Communication between tasks assigned to different platforms incurs an interconnect transfer cost, modeled as an edge-dependent communication delay. Assignments must satisfy a global hardware area constraint. The goal of HW/SW partitioning is to assign tasks to hardware or software to minimize end-to-end execution time under this constraint.

3.1 Problem formulation

We are given a task graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ with $|\mathcal{V}| = n$ nodes and $|\mathcal{E}| = m$ edges, modeled as a directed acyclic graph (DAG), as shown in Figure 1. Each node $i \in \mathcal{V}$ has software and hardware execution costs t_i^{sw} and t_i^{hw} , respectively, and requires hardware area A_i if assigned to hardware. Each edge $(j, i) \in \mathcal{E}$ represents a precedence dependency from task j to task i , with communication cost C_{ji} incurred when the two endpoint tasks are assigned to different platforms.

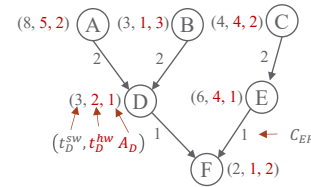


Figure 1: Sample task graph for HW/SW partitioning. Each node shows its software cost, hardware cost, and required hardware area. Directed edges indicate task dependencies, and edge labels represent communication cost when connected tasks are assigned to different types.

We represent a discrete HW/SW partition using a binary vector $\mathbf{p} \in \{0, 1\}^n$, where $p_i = 1$ denotes hardware assignment and $p_i = 0$ denotes software assignment for node i . Since software tasks share a single processor, the final makespan also depends on their execution order. Let π^{sw} denote a valid rank-based order over software tasks, where

$$\pi_i^{sw} < \pi_j^{sw} \implies i \prec_{\pi^{sw}} j, \quad (1)$$

meaning that task i is scheduled before task j on the software processor. Given an assignment $\mathbf{p}$ and software order π^{sw} , let $(\mathbf{s}, \mathbf{f}) \leftarrow \text{Sched}(\mathbf{p}, \pi^{sw})$ denote the induced schedule that returns the corresponding start-time and finish-time vectors $\mathbf{s}$ and $\mathbf{f}$. The

makespan under assignment $\mathbf{p}$ and software order π^{sw} is defined as

$$M(\mathbf{p}, \pi^{sw}) = \max_{i \in \mathcal{V}} f_i. \quad (2)$$

Our goal is to minimize makespan under the hardware area budget:

$$\min_{\mathbf{p} \in \{0,1\}^n, \pi^{sw}} M(\mathbf{p}, \pi^{sw}) = \max_{i \in \mathcal{V}} f_i \quad (3)$$

$$\text{s.t. } (\mathbf{s}, \mathbf{f}) = \text{Sched}(\mathbf{p}, \pi^{sw}), \quad (4)$$

$$\sum_{i \in \mathcal{V}} p_i A_i \leq A_{\max}, \quad (5)$$

$$f_i = s_i + \underbrace{p_i t_i^{hw} + (1 - p_i) t_i^{sw}}_{\text{Execution}}, \quad \forall i \in \mathcal{V}, \quad (6)$$

$$s_i \geq f_j + \underbrace{\mathbb{I}[p_j \neq p_i] C_{ji}}_{\text{Communication}}, \quad \forall (j, i) \in \mathcal{E}, \quad (7)$$

$$s_j \geq f_i, \quad \forall i, j \in \mathcal{V} : p_i = p_j = 0 \text{ and } i \prec_{\pi^{sw}} j, \quad (8)$$

$$s_i \geq 0, \quad \forall i \in \mathcal{V}. \quad (9)$$

Equation (6) defines the finish time of each task under the chosen assignment. Equation (7) enforces precedence constraints together with inter-platform communication delay. Equation (8) enforces sequential execution of software-assigned tasks according to the order π^{sw} . Hardware-assigned tasks may execute in parallel as long as the precedence constraints are satisfied.

3.2 Differentiable Relaxation of the Discrete Problem

Directly optimizing Equations (3)–(9) is difficult because both the placement vector $\mathbf{p} \in \{0, 1\}^n$ and the software order π^{sw} are discrete. In Diff-GNN, we therefore relax the binary assignment vector $\mathbf{p}$ to a continuous vector $\tilde{\mathbf{p}} \in [0, 1]^n$, where $\tilde{p}_i$ denotes the confidence of assigning task i to hardware. To approximate the discrete software order π^{sw} , the GNN also predicts a continuous software-priority vector $\tilde{\pi}^{sw} \in [-1, 1]^n$. These values encode relative precedence among software tasks: if $\tilde{\pi}_i^{sw} > \tilde{\pi}_j^{sw}$, then the relaxed model prefers $i \prec j$. Sorting $\tilde{\pi}^{sw}$ induces a discrete rank-based order that approximates π^{sw} .

Under this relaxation, the execution-time expression in Equation (6) becomes

$$\tilde{t}_i(\tilde{\mathbf{p}}) = \tilde{p}_i t_i^{hw} + (1 - \tilde{p}_i) t_i^{sw}, \quad (10)$$

and the communication term in Equation (7) is approximated by

$$\tilde{c}_{ji}(\tilde{\mathbf{p}}) = |\tilde{p}_j - \tilde{p}_i| C_{ji}. \quad (11)$$

Similarly, instead of the discrete schedule $(\mathbf{s}, \mathbf{f}) = \text{Sched}(\mathbf{p}, \pi^{sw})$, we define a differentiable surrogate scheduler

$$(\tilde{\mathbf{s}}, \tilde{\mathbf{f}}) = \widetilde{\text{Sched}}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}), \quad (12)$$

which approximates the precedence, communication, and software-serialization constraints in Equations (7)–(8).

The max operator appearing in the makespan and readiness computations is not differentiable. We therefore replace it with the

smooth maximum operator

$$\text{SMax}_\beta(\{a_k\}) = \frac{1}{\beta} \log \sum_k e^{\beta a_k}, \quad (13)$$

where $\beta > 0$ controls the sharpness of the approximation. As β increases, SMax_β approaches the standard max operator.

Precedence relation of Equation (7). Let $\text{Pred}(i)$ denote the set of predecessors of node i . The relaxed DAG-readiness term is defined as

$$s_i^{dag} = \begin{cases} 0, & \text{Pred}(i) = \emptyset, \\ \text{SMax}_\beta(\{\tilde{f}_j + \tilde{c}_{ji}(\tilde{\mathbf{p}}) \mid j \in \text{Pred}(i)\}), & \text{otherwise,} \end{cases} \quad (14)$$

which approximates the earliest time at which task i becomes ready from the precedence perspective of Equation (7).

SW relation of Equation (8). To capture serialization on the software processor in Equation (8), we define the relaxed software-readiness term as

$$s_i^{sw} = \text{SMax}_\beta(\{\tilde{f}_j + \phi_{sw}(\tilde{\pi}_j^{sw}, \tilde{\pi}_i^{sw}) \mid j \neq i\}), \quad (15)$$

where $\phi_{sw}(\cdot)$ is a differentiable ordering function that maps the relative priority difference between tasks j and i to a soft serialization penalty. Intuitively, if $\tilde{\pi}_j^{sw}$ indicates that task j should execute before task i , then the contribution of $\tilde{f}_j$ to s_i^{sw} becomes stronger. The explicit construction of ϕ_{sw} is introduced later in the proposed method.

Soft makespan of Equation (2). The relaxed start and finish times are then

$$\tilde{s}_i = \text{SMax}_\beta(\{s_i^{dag}, s_i^{sw}\}), \quad (16)$$

$$\tilde{f}_i = \tilde{s}_i + \tilde{t}_i(\tilde{\mathbf{p}}), \quad (17)$$

which mirror the discrete scheduling relations in Equations (6)–(8).

Let $\text{Sink}(\mathcal{G})$ denote the set of sink nodes in $\mathcal{G}$. The differentiable surrogate makespan is defined as

$$\tilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) = \text{SMax}_\beta(\{\tilde{f}_i \mid i \in \text{Sink}(\mathcal{G})\}). \quad (18)$$

Area of Equation (5). To encourage feasibility under the hardware budget in Equation (5), we use the differentiable area penalty

$$\mathcal{L}_{\text{area}}(\tilde{\mathbf{p}}) = \left[\max \left(0, \sum_{i \in \mathcal{V}} \tilde{p}_i A_i - A_{\max} \right) \right]^2. \quad (19)$$

Relaxed objective. The relaxed objective optimized by Diff-GNN is therefore

$$\mathcal{L}_{\text{relax}}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) = \tilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) + \lambda_a \mathcal{L}_{\text{area}}(\tilde{\mathbf{p}}). \quad (20)$$

Thus, while the original problem minimizes makespan over the discrete placement vector $\mathbf{p}$ and software order π^{sw} , Diff-GNN is trained by minimizing the continuous surrogate in Equation (20). Auxiliary execution, communication, and regularization terms used in practice are introduced later in the proposed method.

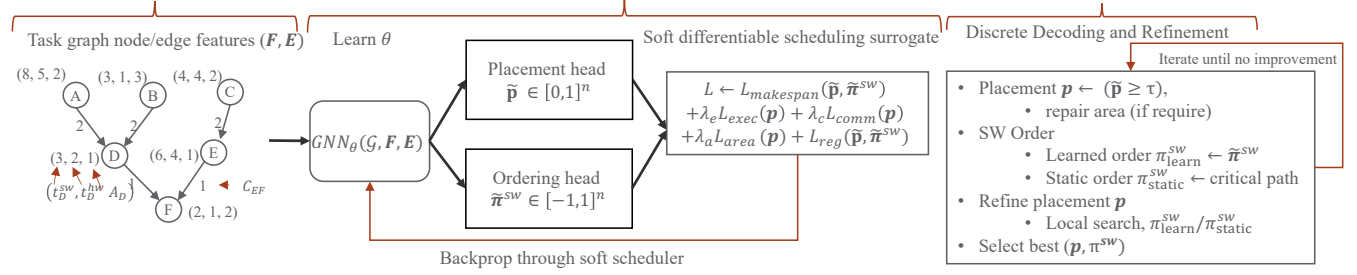


Figure 2: **Sid: placeholder** Overview of Diff-GNN for joint HW/SW partitioning and scheduling. A GNN encoder maps task graph features to the relaxed placement vector $\tilde{\mathbf{p}}$ and relaxed software-priority vector $\tilde{\pi}^{sw}$. A differentiable scheduling surrogate enables end-to-end optimization of makespan under constraints. At inference, solutions are discretized, repaired for feasibility, optionally refined, and evaluated using both the learned software order and a heuristic static-priority order; the better schedule is retained.

4 Proposed Method: Diff-GNN

Building on the relaxed formulation in Section 3, we propose Diff-GNN, a differentiable graph neural framework for jointly optimizing HW/SW partitioning and scheduling. Rather than directly searching over the discrete placement vector $\mathbf{p} \in \{0, 1\}^n$ and software order π^{sw} , Diff-GNN learns a relaxed hardware-assignment vector $\tilde{\mathbf{p}} \in [0, 1]^n$ together with a relaxed software-priority vector $\tilde{\pi}^{sw} \in [-1, 1]^n$, and optimizes the relaxed objective in Equation (20) end-to-end using gradient-based learning.

Figure 2 illustrates the overall pipeline. Given a task graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$, we first construct node and edge features, and a GNN encoder computes node embeddings that are then consumed by two prediction heads: (i) a *placement head* that predicts relaxed hardware-assignment probabilities, and (ii) an *ordering head* that predicts relaxed software priorities. These outputs are passed to an order-aware differentiable scheduler that approximates the makespan while accounting for both precedence constraints and software serialization. During inference, the relaxed solution is discretized, repaired if necessary to satisfy the area constraint, optionally refined, and finally evaluated using a discrete scheduler under two candidate software orders: the learned order induced by $\tilde{\pi}^{sw}$ and a heuristic static-priority order. The better schedule is selected as the final output.

4.1 Input Feature Preparation

Node features. Each node $i \in \mathcal{V}$ is associated with a feature vector capturing execution cost, resource usage, and structural properties:

$$\mathbf{f}_i^{\text{raw}} = \left[t_i^{sw}, t_i^{hw}, A_i, d_i, \frac{t_i^{hw}}{t_i^{sw} + \epsilon}, t_i^{sw} - t_i^{hw}, \frac{t_i^{sw} - t_i^{hw}}{\max(A_i, \epsilon)} \right] \in \mathbb{R}^7. \quad (21)$$

Stacking all node features yields

$$\mathbf{F}^{\text{raw}} = \begin{bmatrix} (\mathbf{f}_1^{\text{raw}})^{\top} \\ \vdots \\ (\mathbf{f}_n^{\text{raw}})^{\top} \end{bmatrix} \in \mathbb{R}^{n \times 7}. \quad (22)$$

We normalize features across nodes using column-wise z-score normalization:

$$\tilde{\mathbf{F}} = \text{Norm}(\mathbf{F}^{\text{raw}}). \quad (23)$$

To incorporate the global hardware-area constraint, we append a constant feature:

$$\mathbf{F} = [\tilde{\mathbf{F}} \quad \rho \mathbf{1}] \in \mathbb{R}^{n \times 8}, \quad (24)$$

where $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector.

Edge features. For each directed edge $(j, i) \in \mathcal{E}$, we define an edge feature vector:

$$\mathbf{e}_{ji}^{\text{raw}} = [C_{ji}, |g_j - g_i|, |ga_j - ga_i|] \in \mathbb{R}_{\geq 0}^3. \quad (25)$$

Stacking all edges yields

$$\mathbf{E}^{\text{raw}} \in \mathbb{R}^{|\mathcal{E}| \times 3}. \quad (26)$$

We normalize edge features using column-wise max scaling:

$$\mathbf{E} = \text{Norm}_{\infty}(\mathbf{E}^{\text{raw}}), \quad (27)$$

where $\text{Norm}_{\infty}(\cdot)$ denotes feature-wise normalization by the maximum value.

4.2 GNN Module

Message passing. Let $\mathbf{h}_i^{(0)} = \mathbf{F}_i$: denote the initial node embedding. We use a message-passing GNN with L layers:

$$\mathbf{h}_i^{(\ell+1)} = \phi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \text{AGG}_{j:(j,i) \in \mathcal{E}} \psi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ji} \right) \right), \quad (28)$$

for $\ell = 0, \dots, L-1$, where $\phi^{(\ell)}$ and $\psi^{(\ell)}$ are learnable functions and AGG is a permutation-invariant aggregation operator.

As a concrete instantiation, we use weighted mean aggregation:

$$\text{AGG}_{j:(j,i) \in \mathcal{E}} \psi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ji} \right) = \frac{1}{\alpha} \sum_{j \in \mathcal{N}(i)} \alpha_{\phi}(j, i) \mathbf{W}^{(\ell)} \mathbf{h}_j^{(\ell)},$$

$$\alpha = \sum_{j \in \mathcal{N}(i)} \alpha_{\phi}(j, i) + \epsilon,$$

where $\mathcal{N}(i)$ denotes the set of incoming neighbors of node i , $\mathbf{W}^{(\ell)}$ is a learnable linear transformation, and $\alpha_{\phi}(j, i) \geq 0$ is a learnable edge weight derived from $\mathbf{e}_{ji}$. We compute

$$\alpha_{\phi}(j, i) = \sigma(f_{\phi}(\mathbf{e}_{ji})), \quad (29)$$

where f_ϕ is an MLP and $\sigma(\cdot)$ is the sigmoid function.

Placement head. From the final embedding $\mathbf{h}_i^{(L)}$, we predict relaxed hardware-assignment probabilities:

$$\tilde{p}_i = \sigma(\text{MLP}_{\text{place}}(\mathbf{h}_i^{(L)})), \quad (30)$$

where $\sigma(\cdot)$ denotes the sigmoid function, ensuring $\tilde{p}_i \in (0, 1)$. Stacking all node-wise predictions gives

$$\tilde{\mathbf{p}} = [\tilde{p}_1, \dots, \tilde{p}_n]^\top. \quad (31)$$

Ordering head. To model software execution order, we compute bounded relaxed software-priority values:

$$\tilde{\pi}_i^{sw} = \tanh(\text{MLP}_{\text{order}}(\mathbf{h}_i^{(L)})), \quad (32)$$

so that $\tilde{\pi}_i^{sw} \in (-1, 1)$. Stacking them yields

$$\tilde{\boldsymbol{\pi}}^{sw} = [\tilde{\pi}_1^{sw}, \dots, \tilde{\pi}_n^{sw}]^\top \in \mathbb{R}^n. \quad (33)$$

4.3 Differentiable Soft Scheduler

We now instantiate the relaxed makespan term $\tilde{M}(\tilde{\mathbf{p}}, \tilde{\boldsymbol{\pi}}^{sw})$ from Section 3. The scheduler takes as input the relaxed placement vector $\tilde{\mathbf{p}}$ and the relaxed software-priority vector $\tilde{\boldsymbol{\pi}}^{sw}$, and produces approximate start and finish times for all tasks.

The most important intermediate quantity is the effective software-order matrix R^{sw} . Figure 3 illustrates its construction on a 6-node example. Read the figure from left to right:

$$\tilde{\boldsymbol{\pi}}^{sw} \rightarrow \mathbf{P}^{sw} \rightarrow \tilde{\mathbf{r}} \rightarrow \mathbf{B}^{sw} \rightarrow \mathbf{R}^{sw} \rightarrow \tilde{M}(\tilde{\mathbf{p}}, \tilde{\boldsymbol{\pi}}^{sw}).$$

First, the ordering head predicts relaxed software priorities $\tilde{\boldsymbol{\pi}}^{sw}$. These values are converted into a soft permutation matrix $\mathbf{P}^{sw}$ using Sinkhorn normalization; $\mathbf{P}^{sw}$ is then mapped to expected ranks $\tilde{r}_i$; the ranks are converted into a pairwise before matrix $\mathbf{B}^{sw}$; and finally $\mathbf{B}^{sw}$ is gated by the software co-assignment matrix so that ordering affects only tasks that are likely mapped to software. The same figure then shows how the resulting order propagates to the relaxed start and finish times and, ultimately, to the relaxed makespan.

From relaxed software priorities to the effective software-order matrix. The ordering head outputs

$$\tilde{\boldsymbol{\pi}}^{sw} = [\tilde{\pi}_1^{sw}, \dots, \tilde{\pi}_n^{sw}]^\top. \quad (34)$$

We first construct a score-to-position compatibility matrix

$$L_{ai} = -\frac{(\tilde{\pi}_i^{sw} - u_a)^2}{\tau_o}, \quad u_a = 1 - \frac{2a}{n-1}, \quad a = 0, \dots, n-1, \quad (35)$$

where $\{u_a\}_{a=0}^{n-1}$ are evenly spaced position anchors and τ_o controls the sharpness of the relaxation. Applying Sinkhorn normalization gives

$$\mathbf{P}^{sw} = \text{Sinkhorn}(L) \in [0, 1]^{n \times n}, \quad (36)$$

where $\mathbf{P}_{ai}^{sw}$ is the soft probability that task i occupies position a in the software order.

Next we compute the expected rank of each task:

$$\tilde{r}_i = \sum_{a=0}^{n-1} a \mathbf{P}_{ai}^{sw}. \quad (37)$$

A smaller $\tilde{r}_i$ indicates that task i is predicted to execute earlier on software. We then convert these ranks into pairwise precedence scores:

$$B_{ji}^{sw} = \sigma\left(\frac{\tilde{r}_i - \tilde{r}_j}{\tau_r}\right), \quad B_{ii}^{sw} = 0, \quad (38)$$

so that B_{ji}^{sw} is large when task j is likely to execute before task i .

Since ordering matters only when both tasks are likely assigned to software, we gate the pairwise precedence by

$$G^{sw} = (1 - \tilde{\mathbf{p}})(1 - \tilde{\mathbf{p}})^\top, \quad (39)$$

and define the effective software-order matrix

$$R_{ji}^{sw} = B_{ji}^{sw}(1 - \tilde{p}_j)(1 - \tilde{p}_i) = B_{ji}^{sw}G_{ji}^{sw}. \quad (40)$$

Thus, R_{ji}^{sw} is large only when task j is likely to precede task i and both tasks are likely mapped to software. In the example of Figure 3, this gives large entries such as $R_{CA}^{sw} = 0.878$, $R_{CB}^{sw} = 0.902$, and $R_{AB}^{sw} = 0.902$, which strongly favor the software order $C \prec A \prec B$.

Relaxed execution and communication. The relaxed execution time of node i is

$$\tilde{t}_i(\tilde{\mathbf{p}}) = \tilde{p}_i t_i^{hw} + (1 - \tilde{p}_i) t_i^{sw}, \quad (41)$$

and the relaxed communication delay on edge (j, i) is

$$\tilde{c}_{ji}(\tilde{\mathbf{p}}) = |\tilde{p}_j - \tilde{p}_i| C_{ji}. \quad (42)$$

DAG readiness. Let $\text{Pred}(i)$ denote the set of predecessors of node i . The DAG-readiness term is

$$s_i^{dag} = \begin{cases} 0, & \text{Pred}(i) = \emptyset, \\ \text{SMax}_\beta(\{\tilde{f}_j + \tilde{c}_{ji}(\tilde{\mathbf{p}}) \mid j \in \text{Pred}(i)\}), & \text{otherwise,} \end{cases} \quad (43)$$

where

$$\text{SMax}_\beta(\{a_k\}) = \frac{1}{\beta} \log \sum_k e^{\beta a_k}. \quad (44)$$

Intuitively, s_i^{dag} is the earliest soft time at which task i becomes ready from the precedence perspective.

Software readiness. Because software tasks execute sequentially on a single CPU, task i may also need to wait for software tasks predicted to execute before it. We model this using R^{sw} :

$$\phi_{sw}(\tilde{\pi}_j^{sw}, \tilde{\pi}_i^{sw}) = \alpha \log(R_{ji}^{sw} + \epsilon), \quad (45)$$

and define the software-readiness term as

$$s_i^{sw} = \text{SMax}_\beta(\{\tilde{f}_j + \phi_{sw}(\tilde{\pi}_j^{sw}, \tilde{\pi}_i^{sw}) \mid j \neq i\}). \quad (46)$$

If R_{ji}^{sw} is large, then task j is likely to delay task i on the software processor.

Start and finish times. The relaxed start time of node i is obtained by combining DAG and software readiness:

$$\tilde{s}_i = \text{SMax}_\beta(\{s_i^{dag}, s_i^{sw}\}), \quad (47)$$

and the corresponding relaxed finish time is

$$\tilde{f}_i = \tilde{s}_i + \tilde{t}_i(\tilde{\mathbf{p}}). \quad (48)$$

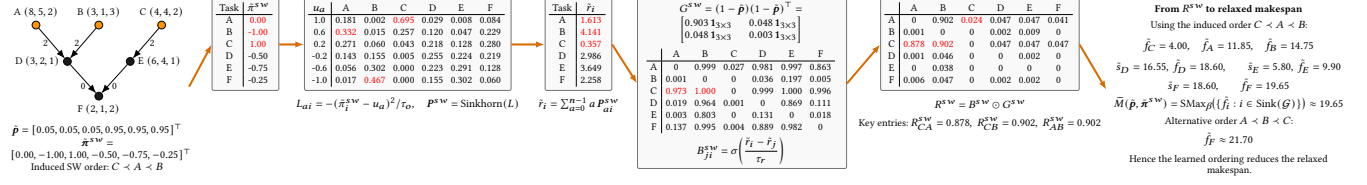


Figure 3: Numerical walk-through of the soft scheduler on a 6-node example. The example graph and relaxed inputs are followed by the ordering vector, Sinkhorn soft permutation matrix, expected ranks, gated rank-based precedence structure, effective software-order matrix R^{sw} , and the resulting relaxed makespan computation. The highlighted entries show that the example strongly favors the software order $C < A < B$.

Soft makespan. Let $\text{Sink}(\mathcal{G})$ denote the set of sink nodes in $\mathcal{G}$. The relaxed makespan is

$$L_{\text{soft-makespan}}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) = \tilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) = \text{SMax}_{\beta}(\{ \tilde{f}_i \mid i \in \text{Sink}(\mathcal{G}) \}). \quad (49)$$

Figure 3 also shows why the induced order $C < A < B$ is useful in this example: it starts the $C \rightarrow E$ branch earlier, which reduces the final relaxed makespan. Appendix ?? provides the full numerical walk-through of the resulting start and finish times.

4.4 Training Objective

The base relaxed objective is given in Equation (20). In practice, we augment it with auxiliary execution, communication, and regularization terms:

$$L = L_{\text{makespan}}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) + \lambda_e L_{\text{exec}}(\tilde{\mathbf{p}}) + \lambda_c L_{\text{comm}}(\tilde{\mathbf{p}}) + \lambda_a L_{\text{area}}(\tilde{\mathbf{p}}) + L_{\text{reg}}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}), \quad (50)$$

Here, L_{makespan} captures the differentiable scheduling objective, while L_{exec} and L_{comm} correspond to relaxed execution and communication costs, respectively.

Execution and communication losses. We directly treat the relaxed execution and communication terms as auxiliary losses:

$$L_{\text{exec}}(\tilde{\mathbf{p}}) = \sum_{i \in \mathcal{V}} \tilde{t}_i(\tilde{\mathbf{p}}), \quad L_{\text{comm}}(\tilde{\mathbf{p}}) = \sum_{(j,i) \in \mathcal{E}} \tilde{c}_{ji}(\tilde{\mathbf{p}}). \quad (51)$$

The execution and communication terms regularize the optimization by discouraging slow assignments and excessive cross-partition communication, thereby aiding makespan minimization.

Area constraint. To enforce the hardware budget during training, we replace the hard constraint with a differentiable hinge penalty:

$$L_{\text{area}}(\tilde{\mathbf{p}}) = \left[\max \left(0, \sum_{i \in \mathcal{V}} \tilde{p}_i A_i - A_{\text{max}} \right) \right]^2. \quad (52)$$

Regularization. We further introduce optional regularization to stabilize ordering predictions and encourage nearly discrete assignments:

$$L_{\text{reg}}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) = \lambda_b L_{\text{binary}}(\tilde{\mathbf{p}}) + \lambda_p L_{\text{perm}}(\mathbf{P}^{sw}), \quad (53)$$

where

$$L_{\text{binary}}(\tilde{\mathbf{p}}) = \frac{1}{n} \sum_{i=1}^n \tilde{p}_i (1 - \tilde{p}_i), \quad (54)$$

$$L_p(\mathbf{P}^{sw}) = \|\mathbf{P}^{sw} \mathbf{1} - \mathbf{1}\|_2^2 + \|(\mathbf{P}^{sw})^\top \mathbf{1} - \mathbf{1}\|_2^2, \quad (55)$$

The term L_{binary} pushes relaxed assignments toward discrete decisions. The term L_{perm} keeps the soft ordering matrix close to a valid permutation relaxation.

4.5 Discrete Inference: Decoding, Scheduling, and Refinement

The outputs of Diff-GNN are converted into a feasible discrete solution through a discrete inference pipeline. This stage bridges the gap between the differentiable training surrogate and the final discrete scheduling objective.

Decoding and candidate generation. Given relaxed placement probabilities $\tilde{\mathbf{p}}$, we generate a set of discrete candidate partitions by thresholding:

$$\hat{p}_i = \mathbb{I}[\tilde{p}_i \geq \tau_d], \quad \tau_d \in \mathcal{T}, \quad (57)$$

where $\mathcal{T}$ is a set of decoding thresholds.

Feasibility repair. Decoded candidates may violate the hardware area constraint. For each candidate, we enforce feasibility:

$$\sum_{i \in \mathcal{V}} \hat{p}_i A_i \leq A_{\text{max}}. \quad (58)$$

When the area constraint is violated, we run a deterministic repair loop: we repeatedly move one task from hardware to software, choosing the hardware task with the lowest benefit-per-area priority. This continues until the hardware area is within budget, thereby guaranteeing feasibility. After that, if area remains, we optionally test software-to-hardware flips that still fit the remaining area and keep the move with the best makespan improvement; by default, only non-worsening moves are accepted.

Software order computation. For each feasible decoded placement $\hat{\mathbf{p}}$, we construct two candidate software orders. The first is the *learned order* π_{learn}^{sw} , obtained by sorting the relaxed software-priority vector $\tilde{\pi}^{sw}$. The second is the *static-priority order* π_{static}^{sw} , computed using the same critical-path priority rule as LSSP on the decoded placement $\hat{\mathbf{p}}$. Specifically, let

$$t_i(\hat{\mathbf{p}}) = \hat{p}_i t_i^{hw} + (1 - \hat{p}_i) t_i^{sw}, \quad (59)$$

and

$$\text{comm}_{ij}(\hat{\mathbf{p}}) = \mathbb{I}[\hat{p}_i \neq \hat{p}_j] C_{ij}. \quad (60)$$

The static priority of task i is computed in reverse topological order as

$$\text{Pri}(i) = t_i(\hat{p}) + \max_{j \in \text{Succ}(i)} (\text{comm}_{ij}(\hat{p}) + \text{Pri}(j)), \quad (61)$$

with $\text{Pri}(i) = t_i(\hat{p})$ for sink tasks. Intuitively, $\text{Pri}(i)$ estimates how critical task i is to the final makespan. Figure 4 illustrates how these priorities are computed bottom-up. If there are multiple sink nodes, we add a dummy sink node and start the computation from there.

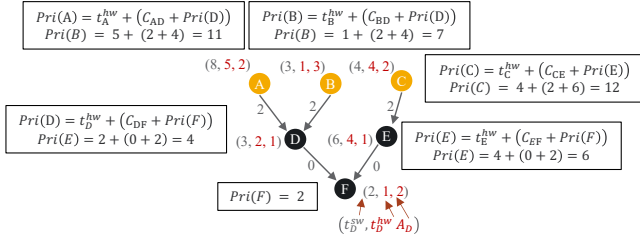


Figure 4: Sid: placeholder Example of bottom-up static-priority computation.

Makespan computation. Given a discrete placement $\hat{p}$, we run list scheduling twice, once with software order $\pi_{\text{learn}}^{\text{sw}}$ and once with $\pi_{\text{static}}^{\text{sw}}$. Let $k \in \{\text{learn}, \text{static}\}$ denote the chosen software-order policy. At each scheduling event t , the next software task is selected from the ready software set

$$\mathcal{R}_{\text{sw}}(t) = \{i \mid \hat{p}_i = 0, \text{Pred}(i) \subseteq C(t)\}, \quad i_t^{(k)} = \arg \max_{i \in \mathcal{R}_{\text{sw}}(t)} \pi_k^{\text{sw}}(i), \quad (62)$$

with deterministic tie-breaking. Dependency readiness is

$$r_i^{(k)} = \max_{j \in \text{Pred}(i)} (f_j^{(k)} + c_{ji}(\hat{p})), \quad (63)$$

and start/finish times are

$$s_i^{(k)} = \max(r_i^{(k)}, t_{i,\text{avail}}^{(k)}), \quad f_i^{(k)} = s_i^{(k)} + t_i(\hat{p}), \quad (64)$$

where

$$t_{i,\text{avail}}^{(k)} = \begin{cases} t_{\text{SW},\text{avail}}^{(k)}, & \hat{p}_i = 0, \\ 0, & \hat{p}_i = 1, \end{cases} \quad (65)$$

and $t_{\text{SW},\text{avail}}^{(k)}$ is the current availability time of the single software lane, updated after each scheduled software task.

This yields two candidate makespans,

$$T^{\text{learn}} = \max_i f_i^{(\text{learn})}, \quad T^{\text{static}} = \max_i f_i^{(\text{static})}. \quad (66)$$

For Diff-GNN, we define the discrete evaluation score of placement $\hat{p}$ as

$$T^*(\hat{p}) = \min(T^{\text{learn}}, T^{\text{static}}), \quad (67)$$

and denote the corresponding selected software order by $\hat{\pi}^{\text{sw}}$.

Optional local refinement. Starting from the repaired placement, we optionally apply local search to explore local changes, such as flipping assignments of critical nodes. Each proposed move is evaluated using the discrete makespan score $T^*(\hat{p})$, and a move is accepted only if it improves the current solution under this evaluation.

Final selection. Across all decoded candidates and their refined versions, we select the solution with the smallest final discrete makespan:

$$(\mathbf{p}^*, \pi^{\text{sw},*}) = \arg \min_{(\hat{p}, \hat{\pi}^{\text{sw}}) \in C} M(\hat{p}, \hat{\pi}^{\text{sw}}), \quad (68)$$

where C denotes the set of all feasible decoded or refined placements together with their selected software orders. Equivalently, the final makespan reported for Diff-GNN is the minimum value of $T^*(\hat{p})$ over all candidates.

For competing algorithms, we use only the LS-based static-priority order, so that all baselines are evaluated under the same standard discrete scheduling rule.

4.6 Pseudocode of Diff-GNN

Algorithms 1 and 2 summarize the training and discrete inference procedures of Diff-GNN.

Algorithm 1: Training of Diff-GNN

Input: Task graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, epochs T , area budget A_{max}
Input: Loss weights $\lambda_e, \lambda_c, \lambda_a, \lambda_b, \lambda_p$
Input: Temperatures τ_o, τ_r and smooth-max parameter β

- 1 Initialize model parameters θ
- 2 Construct node and edge features F, E
- 3 **for** $t = 1, \dots, T$ **do**
- 4 Predict $\hat{p}, \hat{\pi}^{\text{sw}} \leftarrow \text{GNN}_{\theta}(\mathcal{G}, F, E)$
- 5 Build soft precedence from $\hat{\pi}^{\text{sw}}$ using Sinkhorn relaxation
- 6 Compute relaxed execution/communication terms $\tilde{t}_i, \tilde{c}_{ji}$
- 7 Run differentiable scheduler to obtain $\tilde{s}, \tilde{f}$
- 8 Compute $L_{\text{makespan}}, L_{\text{exec}}, L_{\text{comm}}, L_{\text{area}}, L_{\text{reg}}$
- 9 $L \leftarrow L_{\text{makespan}} + \lambda_e L_{\text{exec}} + \lambda_c L_{\text{comm}} + \lambda_a L_{\text{area}} + L_{\text{reg}}$
- 10 Update θ via backpropagation

Algorithm 2: Discrete Inference and Post-Processing

Input: Graph $\mathcal{G}$, trained model, area budget A_{max} , thresholds $\mathcal{T}$
Output: Best discrete placement $\mathbf{p}^*$, software order $\pi^{\text{sw},*}$, makespan M^*

- 1 Compute $\hat{p}, \hat{\pi}^{\text{sw}}$
- 2 Initialize $\mathbf{p}^* \leftarrow \emptyset, \pi^{\text{sw},*} \leftarrow \emptyset, M^* \leftarrow +\infty$
- 3 **for** $\tau_d \in \mathcal{T}$ **do**
- 4 Decode $\hat{p}$ by thresholding: $\hat{p}_i \leftarrow \mathbb{I}[\hat{p}_i \geq \tau_d]$
- 5 Repair $\hat{p}$ until $\sum_i \hat{p}_i A_i \leq A_{\text{max}}$
- 6 Refine $\hat{p}$ using local search // optional
- 7 Compute learned software order $\pi_{\text{learn}}^{\text{sw}}$ by sorting $\hat{\pi}^{\text{sw}}$
- 8 Compute static-priority order $\pi_{\text{static}}^{\text{sw}}$ on $\hat{p}$
- 9 $T^{\text{learn}} \leftarrow M(\hat{p}, \pi_{\text{learn}}^{\text{sw}}), T^{\text{static}} \leftarrow M(\hat{p}, \pi_{\text{static}}^{\text{sw}})$
- 10 $\hat{\pi}^{\text{sw}} \leftarrow \arg \min_{\pi \in \{\pi_{\text{learn}}^{\text{sw}}, \pi_{\text{static}}^{\text{sw}}\}} M(\hat{p}, \pi)$
- 11 $T^*(\hat{p}) \leftarrow \min(T^{\text{learn}}, T^{\text{static}})$
- 12 **if** $T^*(\hat{p}) < M^*$ **then**
- 13 $\mathbf{p}^* \leftarrow \hat{p}, \pi^{\text{sw},*} \leftarrow \hat{\pi}^{\text{sw}}, M^* \leftarrow T^*(\hat{p})$
- 14 **return** $(\mathbf{p}^*, \pi^{\text{sw},*}, M^*)$

4.7 Complexity Analysis

Training Complexity. Let $n = |\mathcal{V}|$, $m = |\mathcal{E}|$, hidden size d , and L GNN layers. The GNN and prediction heads incur a cost of $O(L(md + nd^2)) + O(nd)$. Applying Sinkhorn normalization to an $n \times n$ matrix costs $O(T_{\text{sink}}n^2)$, where T_{sink} (typically 10–12) is the number of normalization iterations, and computing rank-based pairwise precedence requires $O(n^2)$. The differentiable scheduler adds $O(m + n^2)$.

Thus, the total time complexity per forward pass is $T_{\text{total}} = O(L(md + nd^2) + m + T_{\text{sink}}n^2 + n^2)$. The space complexity is $S_{\text{total}} = O(Lnd + m + n^2)$, where the n^2 term (dense ordering tensors) is the primary scalability bottleneck.

Post-processing complexity. For each decoded placement, post-processing requires computing the learned software order, the static-priority order, and evaluating discrete schedules under both. The overall cost per decoded placement is $O(m + n \log n)$. Thus, with $|\mathcal{T}|$ decoding thresholds and R local-refinement evaluations, the total post-processing complexity is $O((|\mathcal{T}| + R)(m + n \log n))$. In practice, this is much smaller than the training cost.

Large-graph approximation. For large-scale graphs, we improve scalability by skipping Sinkhorn normalization and directly using ordering logits for pairwise precedence, while restricting software ordering to the top- k high-confidence nodes per task. This reduces pairwise interactions from $O(n^2)$ to $O(nk)$ with $k \ll n$, yielding $T_{\text{total}} = O(L(md + nd^2) + m + nk)$ and $S_{\text{total}} = O(Lnd + m + nk)$.

5 Experiments

In this section, we answer the following questions.

5.1 Dataset and settings

5.2 Makespan comparison

5.3 Ablation Studies

Base GNN, +sw ordering head, +hw ordering head, +post processing, +greedy fixing, +dag/lssp, +edge weight learning, +features, +objective functions

6 Conclusion

References

- [1] Péter Arató, Zoltán Ádám Mann, and András Orbán. 2005. Algorithmic aspects of hardware/software partitioning. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 10, 1 (2005), 136–156.
- [2] Honglei Han, Wenju Liu, Jigang Wu, and Guiyuan Jiang. 2013. Efficient Algorithm for Hardware/Software Partitioning and Scheduling on MPSoC. *J. Comput.* 8, 1 (2013), 61–68.
- [3] Neng Hou, Xiaohu Yan, and Fazhi He. 2019. A survey on partitioning models, solution algorithms and algorithm parallelization for hardware/software co-design. *Design Automation for Embedded Systems* 23, 1 (2019), 57–77.
- [4] Jürgen Teich. 2012. Hardware/software codesign: The past, the present, and predicting the future. *Proc. IEEE* 100, Special Centennial Issue (2012), 1411–1430.
- [5] Alessandro B Trindade and Lucas C Cordeiro. 2016. Applying SMT-based verification to hardware/software partitioning in embedded systems. *Design Automation for Embedded Systems* 20, 1 (2016), 1–19.
- [6] Xin Zheng, Shouzhi Liang, and Xiaoming Xiong. 2021. A hardware/software partitioning method based on graph convolution network. *Design Automation for Embedded Systems* 25, 4 (2021), 325–351.

A Appendix

Notations: Table 1 summarizes the main notations used in this paper.

Table 1: Main notations.

$\mathcal{G}(\mathcal{V}, \mathcal{E})$	$\triangleq$	Task graph
(j, i)	$\triangleq$	Dependency edge $j \rightarrow i$
t_i^{sw}, t_i^{hw}	$\triangleq$	SW/HW execution costs
$A_i, A_{\max}$	$\triangleq$	Node area and area budget
C_{ji}	$\triangleq$	Communication cost on (j, i)
$\mathbf{p}, \tilde{\mathbf{p}}, \hat{\mathbf{p}}$	$\triangleq$	Discrete, relaxed, and decoded placements
$p_i, \tilde{p}_i, \hat{p}_i$	$\triangleq$	Corresponding node assignments
$\pi^{sw}, \tilde{\pi}^{sw}$	$\triangleq$	Discrete and relaxed software order
$\pi_{\text{learn}}^{sw}, \pi_{\text{static}}^{sw}$	$\triangleq$	Learned and static software orders
$\mathbf{s}, \mathbf{f}$	$\triangleq$	Discrete start and finish times
$\tilde{\mathbf{s}}, \tilde{\mathbf{f}}$	$\triangleq$	Relaxed start and finish times
$M(\mathbf{p}, \pi^{sw})$	$\triangleq$	Discrete makespan
$\tilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw})$	$\triangleq$	Relaxed makespan
$\tilde{t}_i(\tilde{\mathbf{p}}), \tilde{c}_{ji}(\tilde{\mathbf{p}})$	$\triangleq$	Relaxed execution and communication
$\text{SMax}_{\beta}(\cdot)$	$\triangleq$	Smooth maximum